

PRÁCTICA GUIADA – ETAPA 5

PROYECTO "SERVICIOS TÉCNICOS"

Primera API REST y módulo de Clientes

Flask + MySQL + React + JavaScript + CSS + Git/GitHub

1. Propósito

Desarrollar la primera funcionalidad real del sistema Servicios Técnicos: un módulo de Clientes que consulte información de MySQL mediante una API REST en Flask y muestre los registros en React. Se preparará además la estructura para agregar registros.

2. Objetivos

- Crear una API REST en Flask.
- Crear un endpoint GET para consultar MySQL.
- Probar la API con Thunder Client.
- Crear un servicio de API en React.
- Mostrar registros en una tabla.
- Agregar la ruta Clientes al sistema.
- Proteger /clientes con RutaProtegida.
- Registrar cambios con Git y GitHub.

3. Requisitos previos

- Etapa 4 terminada.
- Login y Dashboard funcionando.
- RutaProtegida funcionando.
- Flask en http://127.0.0.1:5000.
- React en http://localhost:5173/.
- MySQL y base tecnico disponibles.



4. Verificar la estructura real de MySQL

En MySQL ejecuta:

```
USE tecnico;
SHOW TABLES;
DESCRIBE cliente;
```

IMPORTANTE: si la tabla o sus columnas tienen otros nombres, utiliza exactamente los nombres mostrados por DESCRIBE. No inventes columnas.

5. Iniciar los servidores

Backend:

```
cd backend
venv\Scripts\activate
python -m app.main

Running on http://127.0.0.1:5000
```

Frontend, en otra terminal:

```
cd frontend
npm run dev

Local: http://localhost:5173/
```

6. Crear la ruta Flask

```
backend/app/routes/clientes.py

from flask import Blueprint, jsonify
from config.conexion import conectar

clientes_bp = Blueprint("clientes", __name__)

@clientes_bp.route("/api/clientes", methods=["GET"])
def obtener_clientes():
    conexion = conectar()
    if conexion is None:
        return jsonify({"success": False, "message": "No se pudo conectar a la base de datos"}),
500
    cursor = None
    try:
        cursor = conexion.cursor(dictionary=True)
        cursor.execute("SELECT * FROM cliente")
```




```
        clientes = cursor.fetchall()
        return jsonify({"success": True, "clientes": clientes}), 200
    except Exception as e:
        return jsonify({"success": False, "message": str(e)}), 500
    finally:
        if cursor:
            cursor.close()
        conexion.close()
```

Si la tabla se llama clientes, cambia SELECT * FROM cliente por SELECT * FROM clientes.

7. Registrar el Blueprint

En backend/app/main.py agrega:

```
from app.routes.clientes import clientes_bp

app.register_blueprint(clientes_bp)
```

Reinicia Flask con Ctrl+C y después:

```
python -m app.main
```

8. Probar GET con Thunder Client

1. Abre Thunder Client en VS Code.
2. New Request → método GET.
3. URL: http://127.0.0.1:5000/api/clientes
4. No agregues Body.
5. Presiona Send.

Una respuesta exitosa tendrá esta forma general:

```
{
  "success": true,
  "clientes": [
    { "id": 1, "nombre": "Juan Pérez" }
  ]
}
```

Los campos reales dependerán de tu tabla.



9. Interpretar resultados

- 200 OK: consulta correcta.
- 404 Not Found: revisar URL y Blueprint.
- 500 Internal Server Error: revisar SQL, tabla y terminal Flask.
- Error de conexión: revisar MySQL y config/conexion.py.

10. Crear el servicio React

En frontend/src/services/api.js agrega:

```
export async function obtenerClientes() {
  const respuesta = await fetch("http://127.0.0.1:5000/api/clientes");
  const datos = await respuesta.json();
  return { ok: respuesta.ok, status: respuesta.status, datos };
}
```

11. Crear Clientes.jsx

```
frontend/src/pages/Clientes.jsx

import { useEffect, useState } from "react";
import { obtenerClientes } from "../services/api";

function Clientes() {
  const [clientes, setClientes] = useState([]);
  const [mensaje, setMensaje] = useState("");

  useEffect(() => { cargarClientes(); }, []);

  const cargarClientes = async () => {
    try {
      const resultado = await obtenerClientes();
      if (resultado.ok && resultado.datos.success) {
        setClientes(resultado.datos.clientes);
      } else {
        setMensaje("No se pudieron obtener los clientes");
      }
    } catch (error) {
      setMensaje("Error al conectar con el servidor");
    }
  };

  return (
    <div className="dashboard">
```




```
        <h1>Clientes</h1>
        {mensaje && <p className="error">{mensaje}</p>}
        <table>
          <thead><tr><th>ID</th><th>Nombre</th></tr></thead>
          <tbody>
            {clientes.map((cliente) => (
              <tr key={cliente.id}>
                <td>{cliente.id}</td>
                <td>{cliente.nombre}</td>
              </tr>
            ))}
          </tbody>
        </table>
      </div>
    );
}
export default Clientes;
```

Adapta cliente.id y cliente.nombre a las columnas reales de tu tabla.

12. Agregar la ruta /clientes

En frontend/src/App.jsx agrega:

```
import Clientes from "./pages/Clientes";

<Route path="/clientes" element={<RutaProtegida><Clientes /></RutaProtegida>} />
```

13. Agregar enlace en Dashboard

En Dashboard.jsx, dentro de nav:

```
<Link to="/clientes">Clientes</Link>
```

14. Flujo completo

```
Login → Dashboard → Clientes → GET /api/clientes → Flask → MySQL → JSON → React → Tabla
```

15. Prueba desde el navegador

6. Inicia Flask.
7. Inicia Vite.
8. Abre http://localhost:5173/.
9. Inicia sesión.
10. Selecciona Clientes.
11. Comprueba que aparezcan los registros.



16. Prueba de protección

Cierra sesión e intenta abrir:

```
http://localhost:5173/clientes
```

Resultado esperado: RutaProtegida redirige al Login.

17. Preparación para POST

La siguiente operación será crear clientes. El patrón será:

```
Formulario React → POST /api/clientes → Flask → INSERT INTO cliente → MySQL → JSON → React
```

No escribas el INSERT hasta conocer las columnas reales mediante DESCRIBE cliente.

18. Errores frecuentes

Problema	Solución
404 /api/clientes	Revisar clientes_bp y register_blueprint.
500	Revisar tabla, columnas y terminal Flask.
No aparecen datos	Comprobar que la tabla tenga registros.
Failed to fetch	Comprobar Flask y CORS.
Cannot find module	Revisar ruta/nombre del archivo.
Clientes no abre	Revisar la ruta /clientes en App.jsx.

19. Tabla de pruebas

Prueba	Acción	Esperado	Resultado	Evidencia
API GET	GET /api/clientes	200 + JSON		
React	Abrir Clientes	Tabla visible		
Protección	Abrir /clientes sin sesión	Login		
Backend apagado	Abrir Clientes	Mensaje de error		



20. Preguntas de análisis

12. ¿Qué es una API REST?

Respuesta: _____

13. ¿Qué diferencia existe entre GET y POST?

Respuesta: _____

14. ¿Qué función cumple un Blueprint?

Respuesta: _____

15. ¿Por qué se utiliza conectar()?

Respuesta: _____

16. ¿Por qué debemos revisar DESCRIBE antes del INSERT?

Respuesta: _____

17. ¿Qué función cumple useEffect?

Respuesta: _____

18. ¿Qué función cumple useState?

Respuesta: _____

19. ¿Por qué React no se conecta directamente a MySQL?

Respuesta: _____

20. ¿Qué ventaja tiene services/api.js?

Respuesta: _____

21. ¿Por qué /clientes debe estar protegida?

Respuesta: _____



21. Git y GitHub

```
git status
git add .
git commit -m "Agrega API y modulo de clientes"
git push
```

.

22. Punto de control

La Etapa 5 está terminada cuando el estudiante puede iniciar sesión, entrar a Clientes, consultar registros de MySQL mediante GET /api/clientes, visualizar la información en React y demostrar que /clientes está protegida.